



Orient BlackSwan

CLUSTERING

CHAPTER 7: MACHINE LEARNING – THEORY & PRACTICE



Universities Press

Introduction

- Clustering refers to the process of arranging or organizing objects according to specific criteria.
- Partitioning of data
 - Grouping of data in database application improves the data access
- Data reorganization

Pattern	f1	f2	f3	f4	f5	f6
1	1	0	1	0	1	0
2	0	1	0	1	0	1
3	1	0	1	0	1	0
4	0	1	0	1	0	1

Row data

Pattern	f1	f2	f3	f4	f5	f6
1	1	0	1	0	1	0
3	1	0	1	0	1	0
2	0	1	0	1	0	1
4	0	1	0	1	0	1

Data re-organization using row as a criteria

- Data compression

Pattern	f1	f2	f3	f4	f5	f6	count
1,3	1	0	1	0	1	0	2
2,4	0	1	0	1	0	1	2

Compressed data

Introduction

- Summarization
 - Statistical measures like mean, mode, and median can provide summarized information.
- Matrix factorization
 - Let there be n data points in an l -dimensional space. We can represent it as a matrix $X_{n \times l}$.
 - It is possible to approximate X as a product of two matrices $B_{n \times K}$ and $C_{K \times l}$.
 - So, $X \approx BC$, where B is the cluster assignment matrix and C is the representatives matrix

Clustering process

- The clustering process ensure that the distance between any two points within the same cluster (intra-cluster distance), as measured by a dissimilarity measure such as Euclidean distance, is smaller than the distance between any two points belonging to different clusters (inter-cluster distance)
- Any two points are placed in the same cluster if the distance between them is lower than a certain threshold (input). Squared Euclidean distance is used as one of the measure to compute the distance between the points.

Hard and soft clustering

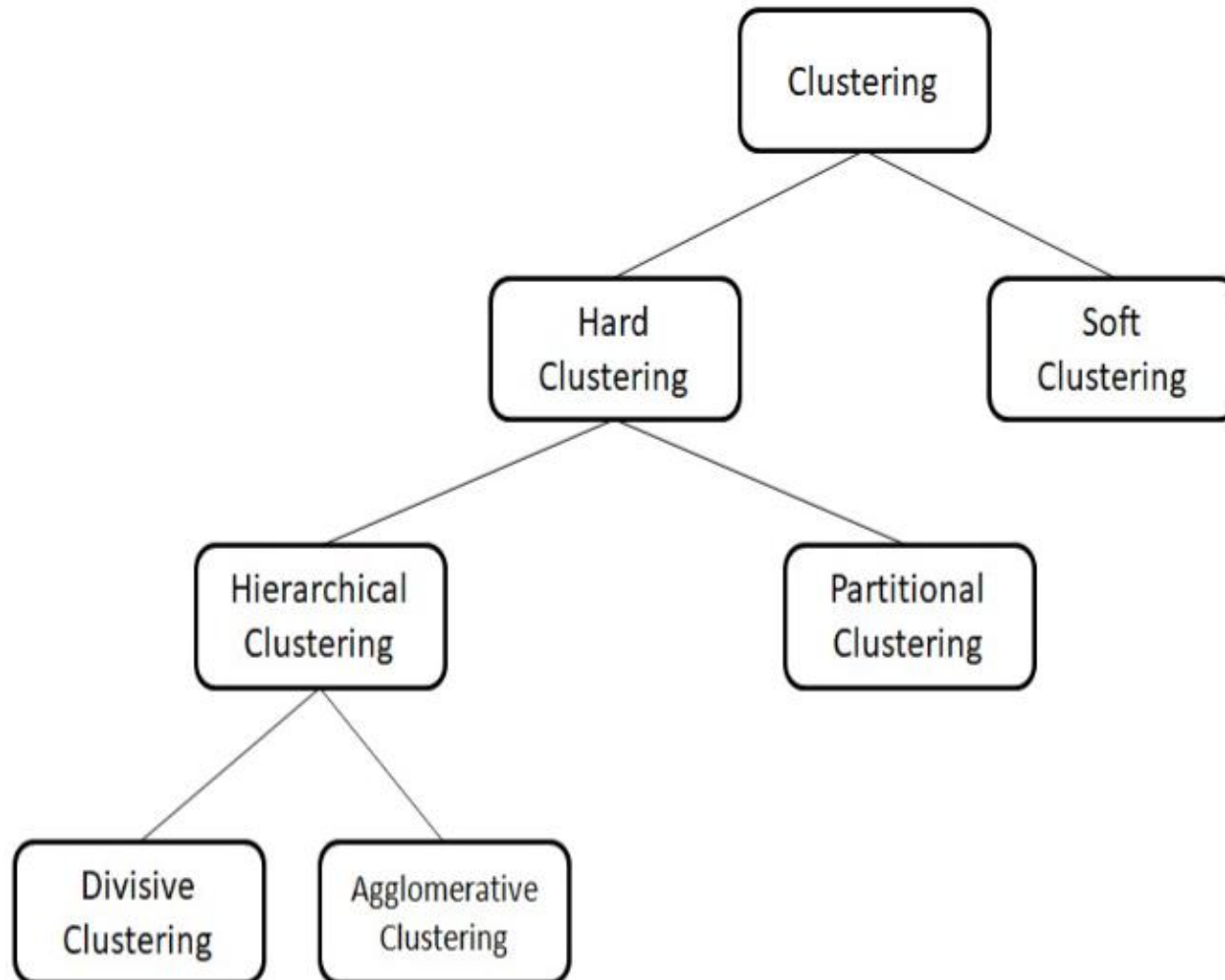
Let $\mathfrak{X} = \{x_1, x_2, \dots, x_n\}$ be n data points which are mapped to C clusters, $\mathfrak{C} = \{X_1, X_2, \dots, X_c\}$ such that

1. $\cup_{i=1}^C X_i = \mathfrak{X}$ and
2. $X_i \neq \phi, \forall i \in \{1, 2, \dots, c\}$.
3. If $\forall i, j, i \neq j, x_i \cap x_j = \phi$, then the clustering is **hard**, else it is **soft**

Data abstraction

- Clustering is a useful method for data abstraction, and it can be applied to generate clusters of data points that can be represented by their centroid or medoid or leader or some other suitable entity.
- The centroid is computed as the sample mean of the data points in a cluster.
- The medoid is the point that minimizes the sum of distances to all other points in the cluster.
- Note: The centroid can shift dramatically based on the position of the outlier, while the medoid remains stable within the boundaries of the original cluster.

Clustering algorithms



Divisive clustering

- Divisive algorithms are either polythetic where the division is based on more than one feature or monothetic when only one feature is considered at a time.
- The polythetic scheme is based on finding all possible 2-partitions of the data and choosing the best among them. If there are n patterns, the number of distinct 2-partitions is given by $(2^n - 2)/2 = 2^{n-1} - 1$.

Divisive clustering

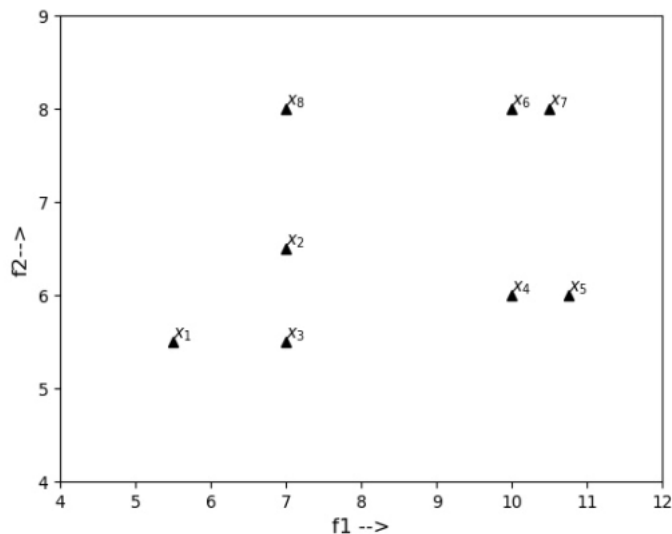
- Among all possible 2-partitions, the partition with the least sum of the sample variances of the two clusters is chosen as the best.
- From the resulting partition, the cluster with the maximum sample variance is selected and is split into an optimal 2-partition.
- This process is repeated till we get singleton clusters.
- If a collection of patterns (data points) is split into two clusters with p patterns $x_1, \dots, x_p$ in one cluster and q patterns $y_1, \dots, y_q$ in the other cluster with the centroids of the two clusters being $C1$ and $C2$ respectively, then the sum of the sample variances will be

$$\sum_{i=1}^p (x_i - C1)^2 + \sum_{j=1}^q (y_j - C2)^2$$

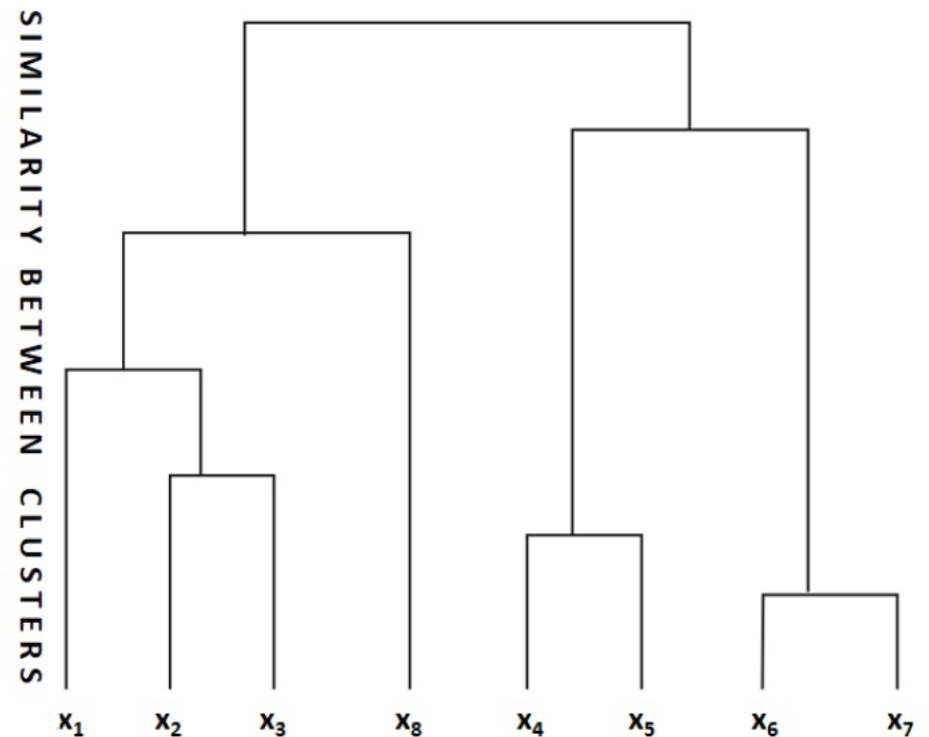
Divisive clustering

Patterns	f1	f2
x_1	5.5	5.5
x_2	7	6.5
x_3	7	5.5
x_4	10	6
x_5	10.75	6
x_6	10	8
x_7	10.5	8
x_8	7	8

Example data patterns with two features



Visual representation of data patterns



Divisive clustering for polythetic clustering for the data points

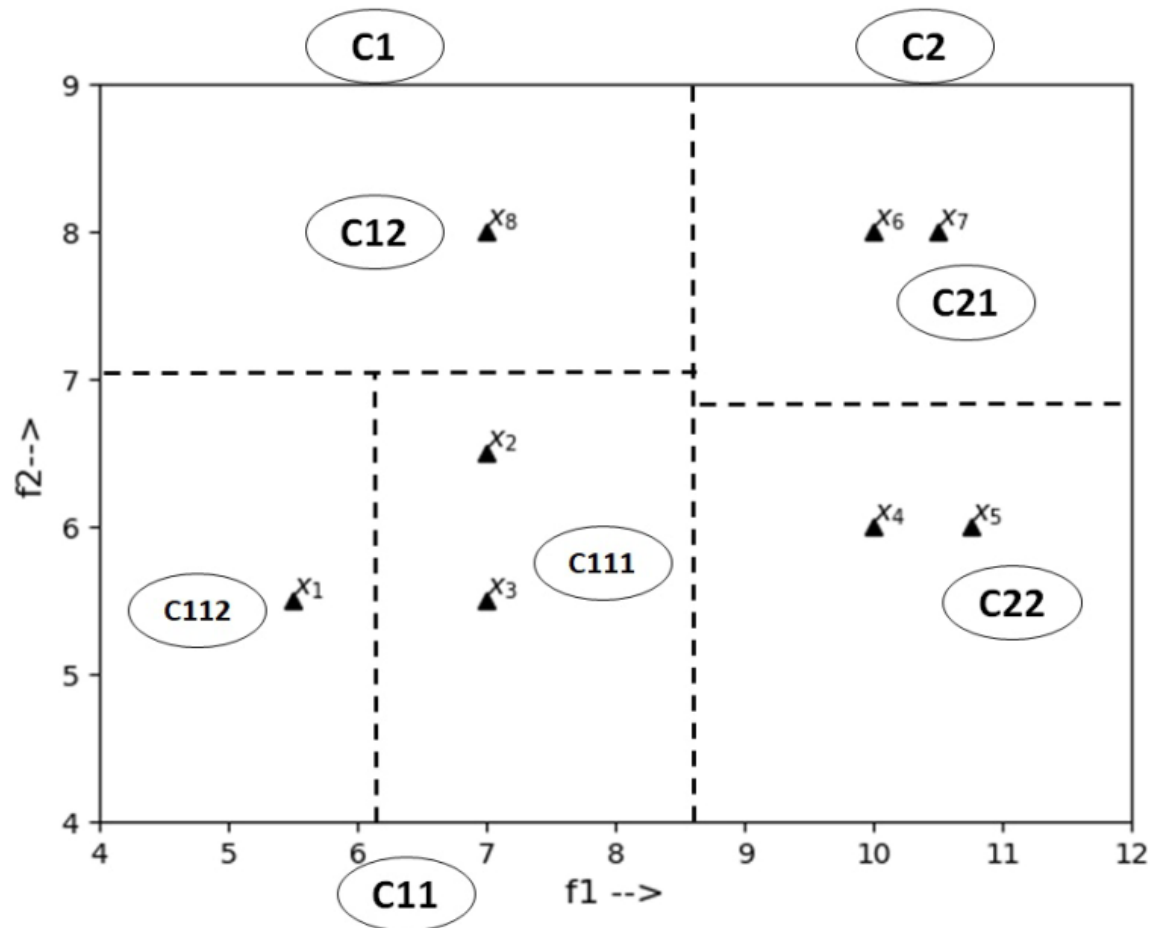
Monothetic clustering

- involves considering each feature direction individually and dividing the data into two clusters based on the gap in projected values along that feature direction.
- Specifically, the dataset is split into two parts at a point that corresponds to the mean value of the maximum gap observed among the feature values.
- This process is then repeated sequentially for the remaining features, further partitioning each cluster.

Monothetic clustering

Patterns	f1	f2
x_1	5.5	5.5
x_2	7	6.5
x_3	7	5.5
x_4	10	6
x_5	10.75	6
x_6	10	8
x_7	10.5	8
x_8	7	8

Example data patterns with two features



Monothetic clustering for the data points

Agglomerative clustering

- An agglomerative clustering algorithm generally follows the following steps:
 1. Compute the proximity matrix for all pairs of patterns in the dataset.
 2. Find the closest pair of clusters based on the computed proximity measure and merge them into a single cluster. Update the proximity matrix to reflect the merge, adjusting the distances between the newly formed cluster and the remaining clusters.
 3. If all patterns belong to a single cluster, terminate the algorithm. Otherwise, go back to Step 2 and repeat the process until all patterns are in one cluster.

Agglomerative clustering

Patterns	f1	f2
x_1	5.5	5.5
x_2	7	6.5
x_3	7	5.5
x_4	10	6
x_5	10.75	6
x_6	10	8
x_7	10.5	8
x_8	7	8

Example data patterns with two features

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8
x_1	0	2.5	1.5	5	5.5	7	7.5	4
x_2	2.5	0	1	3.5	4.25	4.5	5	1.5
x_3	1.5	1	0	3.5	4.25	5.5	6	2.5
x_4	5	3.5	3.5	0	0.75	2	2.5	5
x_5	5.75	4.25	4.25	0.75	0	2.75	2.25	5.75
x_6	7	4.5	5.5	2	2.75	0	0.5	3
x_7	7.5	5	6	2.5	2.25	0.5	0	3.5
x_8	4	1.5	2.5	5	5.75	3	3.5	0

City-block distance between pairs of data points

	x_1	x_2	x_3	x_4	x_5	$C1=\{x_6, x_7\}$	x_8
x_1	0	2.5	1.5	5	5.5	7	4
x_2	2.5	0	1	3.5	4.25	4.5	1.5
x_3	1.5	1	0	3.5	4.25	5.5	2.5
x_4	5	3.5	3.5	0	0.75	2	5
x_5	5.75	4.25	4.25	0.75	0	2.75	5.75
$C1 = \{x_6, x_7\}$	7	4.5	5.5	2	2.25	0	3
x_8	4	1.5	2.5	5	5.75	3	0

City-block distance among the data points after merging x_6 and x_7 as one cluster

k-Means clustering

1. Select k initial cluster centers from the given n data points. The remaining $(n - k)$ data points are then assigned to one of the k clusters based on their proximity to the closest cluster center.
2. Compute the new cluster centers based on the current assignment of data points. This is done by calculating the mean or centroid of all the data points belonging to each cluster.
3. Assign each of the n data points to the cluster center that is closest in proximity to it.
4. Check if there have been any changes in the assignment of data points to clusters. If no changes have occurred, terminate the algorithm; otherwise, go back to Step 2 and repeat the process.

Elbow method to select k

1. Perform the k-means clustering algorithm (as discussed earlier) for different value of k .
2. For each k , calculate the total in-cluster sum of squared error (ic-sse):

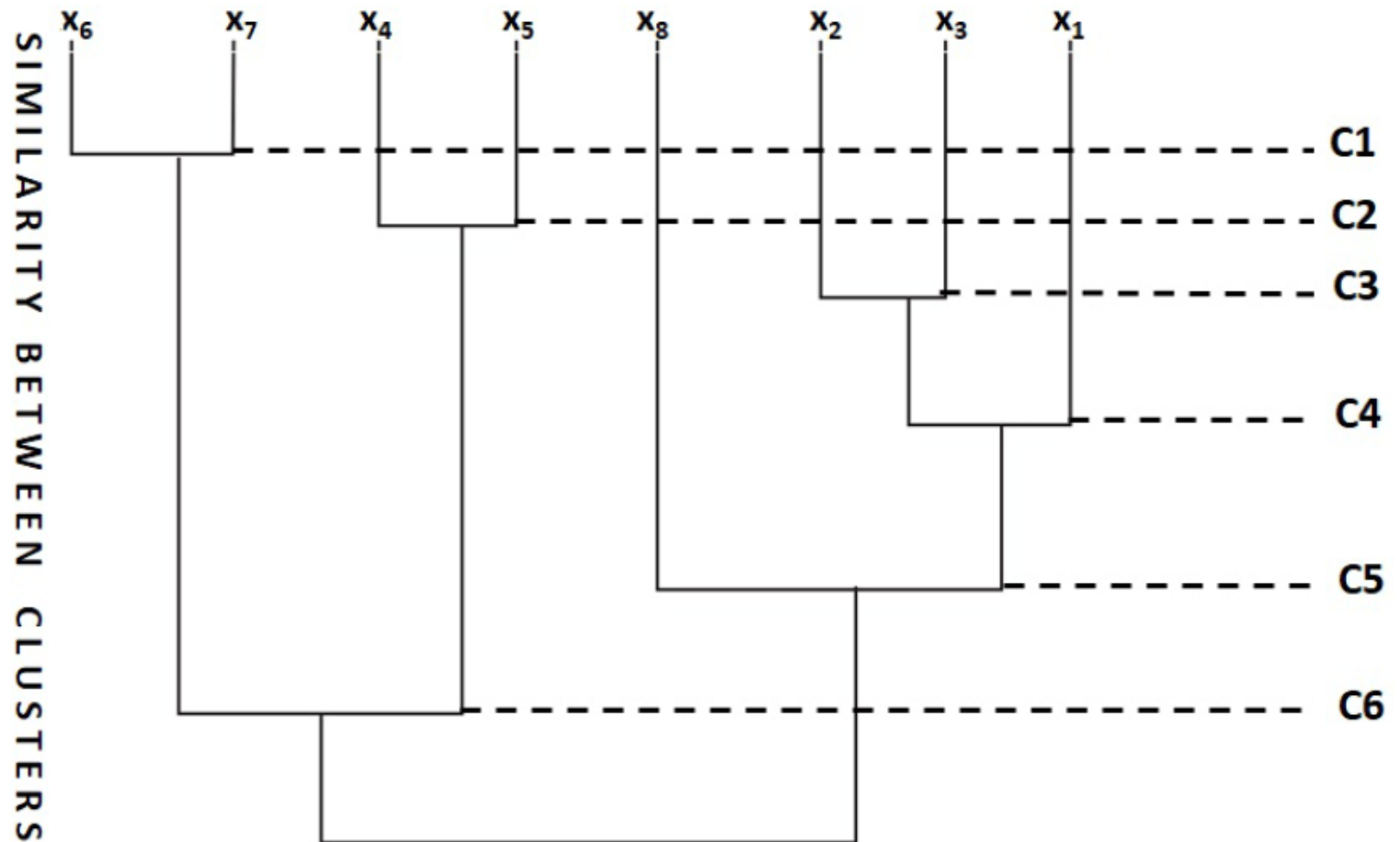
$$\text{ic-sse} = \sum_{i=1}^k \sum_{x \in C_i} (x - \mu_i)^2$$

3. Plot the curve, ic-sse verses k .
4. The location of a bend (knee) in the plot is generally considered as an indicator of an appropriate value for k .

k-Means++ clustering

- k-means++ clustering algorithm is mainly used for identifying the initial cluster centers.
- The steps to identify the initial k centroids are as follows:
 1. Initially, select a random data point as the first centroid.
 2. Compute the distance (say Euclidean distance) between each centroid and the other data points. Record the minimum distances between any centroid and the data point.
 3. Compute the probabilities among the computed minimum distances.
 4. Choose the next centroid with maximum probability among all probabilities computed in step 3.
 5. Repeat steps 2 to 4 till we end up with the k number of centroids.

Agglomerative clustering



Agglomerative clustering for the data points

Soft partitioning

1. Fuzzy clustering: Each data point is assigned to multiple clusters, typically more than one, based on a membership value. The value is computed using the data point and the corresponding cluster centroid.
2. Rough clustering: Each cluster is assumed to have both a non-overlapping part and an overlapping part. Data points in the non-overlapping portion exclusively belong to that cluster, while data points in the overlapping part may belong to multiple clusters.
3. Neural network-based clustering: In this method, varying weights associated with the data points are used to obtain a soft partition.

Soft partitioning – Contd.

1. Simulated annealing: In this case, the current solution is randomly updated, and the resulting solution is accepted with a certain probability. If the resulting solution is better than the current solution, it is accepted; otherwise, it is accepted with a probability ranging from 0 to 1.
2. Tabu search: Unlike simulated annealing, multiple solutions are stored, and the current solution is perturbed in various ways to determine the next configuration.
3. Evolutionary algorithms: This method maintains a population of solutions. In addition to the fitness values of individuals, a random search based on the interaction among solutions with mutation is employed to generate the next population.

Fuzzy c-means clustering

- In this type, each data point x_i can be assigned to a cluster C_j with a membership value μ_{ij} , which represents the degree to which x_i belongs to C_j .
- The membership value μ_{ij} is assumed to range between 0 and 1, and for each data point x_i , the sum of all μ_{ij} values across clusters is equal to 1.
- The fuzzy c-means clustering algorithm follows an iterative scheme similar to the k-means algorithm.
- It begins by identifying c initial cluster centers from the dataset or membership values between each data point and the cluster. The following steps are then performed iteratively:

Fuzzy c-means clustering – contd.

1. *Compute membership values:*

$$\mu_{ij} = \frac{d(x_j, C_i)^{-1/(M-1)}}{\sum_{l=1}^c d(x_j, C_l)^{-1/(M-1)}}$$

2. *Compute fuzzy cluster centroid:*

$$C_i = \frac{\sum_{j=1}^m (\mu_{ij})^M \times x_j}{\sum_{j=1}^m (\mu_{ij})^M}$$

In above expressions

- μ_{ij} represents the membership value of data point x_j in cluster C_i , where $j = 1, 2, \dots, n$ and $i = 1, 2, \dots, c$.
- $d(x_j, C_i)$ denotes the Euclidean distance between x_j and C_i .
- C_i represents the centroid of the i th cluster.
- M is the fuzzyifying constant which determines the behavior of the algorithm. When $M = 1$, the fuzzy algorithm functions similar to the hard k-means algorithm, where $\mu_{ij} = 1$ when X_j is assigned to C_i . As M increases or tends to infinity, μ_{ij} approaches $\frac{1}{c}$, as the exponents in both the numerator and denominator tend towards 0.

3. Repeat the above steps, and the algorithm terminates when there is no significant change in the computed values for membership and cluster centroid.

Rough clustering

- We can define rough set using the lower approximation ($\underline{R}(S)$) and the upper approximation ($\overline{R}(S)$) where $S \subseteq \mathfrak{X}$, a set of data points as follows:

$$\underline{R}(S) = \bigcup_i G_i, \text{ where } G_i \subseteq S$$

$$\overline{R}(S) = \bigcup_i G_i, \text{ where } G_i \cap S \neq \emptyset$$

- In the lower approximation, each equivalence class must be entirely contained within S .
- On the other hand, in the upper approximation, equivalence classes that partially overlap with S are included.
- Consequently, the lower approximation comprises patterns that definitely belong to S , whereas the upper approximation contains patterns that may potentially belong to S .
- Further note that if $\overline{R}(S) - \underline{R}(S) = \emptyset$, then there is no roughness in S with respect to R .
- We discuss one of the popular clustering algorithms, Rough k -means clustering algorithm in the next subsection which is based on these concept.

Rough k-means clustering algorithm

Let n be the number of data points, $\mathfrak{X} = \{x_1, x_2, \dots, x_n\}$ having l features, k be the number of clusters, w_l and w_u are the weights associated with lower and upper approximation, ϵ is the threshold value.

1. Randomly assign each data object to exactly one lower approximation $\underline{R}(k)$. Note that by definition, the data object also belongs to upper approximation $\overline{R}(k)$ of the same cluster.
2. Compute the cluster centroid, C_j as follows:

- If $\underline{R}(k) \neq \emptyset$ and $\overline{R}(k) - \underline{R}(k) = \emptyset$

$$C_j = \sum_{x_i \in \underline{R}(k)} \frac{x_i}{|\underline{R}(k)|}$$

- Else If $\underline{R}(k) = \emptyset$ and $\overline{R}(k) - \underline{R}(k) \neq \emptyset$

$$C_j = \sum_{x_i \in \overline{R}(k) - \underline{R}(k)} \frac{x_i}{|\overline{R}(k) - \underline{R}(k)|}$$

- Else

$$C_j = w_l \times \sum_{x_i \in \underline{R}(k)} \frac{x_i}{|\underline{R}(k)|} + w_u \times \sum_{x_i \in \overline{R}(k) - \underline{R}(k)} \frac{x_i}{|\overline{R}(k) - \underline{R}(k)|}$$

Rough k-means clustering algorithm

3. For each data point in $\mathfrak{X}$, compute Euclidean distance with each cluster, C_j , $1 \leq j \leq k$ [i.e., $d(x_i, C_j)$, $\forall i \in m, \forall j \in k$].
Let $d(X, C_p) = \min_{1 \leq j \leq k} (x, C_j)$.
4. Use the ratio, $\frac{d(x, C_j)}{d(x, C_p)}$, where $1 \leq i, p \leq k$ to determine the membership of X as follows:
 - If the ratio $\not\leq \epsilon$ then the corresponding data point, x will not be part of the lower approximation.
 - Else, the corresponding data point, x will be a part of the lower approximation
5. Repeat steps from 2 until convergence (i.e., Old centroid = New centroid)

Clustering large datasets

- Issues:
 - Number of dataset scans
 - Incremental changes in dataset
- Solutions:
 - Single dataset scan clustering algorithms
 - Incremental clustering algorithms
 - Abstraction based clustering
 - Examples
 - PC-Clustering algorithm
 - Leader clustering algorithm

Divide-and-conquer method

- The divide-and-conquer approach is an effective strategy for addressing the challenge of clustering large datasets that cannot be stored entirely in main memory.
- To overcome this limitation, a common solution is to process a portion of the dataset at a time and store the relevant cluster representatives in memory.

Divide-and-conquer method

1. *Partitioning the data:* The dataset, which has a size of $n \times l$, is divided into p blocks. The optimal value of p can be determined based on the chosen clustering algorithm and other factors.
2. *Processing each block:* Each block is transferred from secondary storage to main memory for clustering. A standard clustering algorithm is applied to each block, resulting in k clusters.
3. *Representative selection:* A representative pattern for each of the k clusters is identified, resulting in pk representatives over all the p blocks.
4. *Clustering the representatives:* The pk representatives are clustered into k clusters, potentially using the same or a different clustering algorithm. This step helps to organize and group the representative patterns.
5. *Re-labeling the original data:* The cluster labels of the representative patterns obtained in the previous step are used to re-label the original dataset. This ensures that each original data point is assigned to the appropriate cluster based on the clustering of the representatives.

Divide-and-conquer method

